Security Review — Deployment Portal

Version: v2.1.1 **Date:** 2026-03-31 **Primary URL:** https://deployment-portal-instrumental.web.app/ (Firebase Hosting — no IAP) **Legacy URL:** https://deployment-portal-901459055521.us-central1.run.app/ (Cloud Run + Google IAP) **Stack:** React 18 + Firebase Realtime Database + Firebase Hosting + Google OAuth

---

Deployment Architecture Overview

The portal currently has two live URLs serving the same codebase and Firebase backend:

	Firebase Hosting (Primary)	Cloud Run (Legacy)
URL	`deployment-portal-instrumental.web.app`	`deployment-portal-901459055521.us-central1.run.app`
IAP	❌ None	✅ Google Identity-Aware Proxy
Access gate	Login page publicly reachable	Blocked at infra level before page loads
Data backend	Firebase Realtime Database (shared)	Firebase Realtime Database (shared)
Deploy method	`firebase deploy`	Docker + `gcloud run deploy`
Status	Active — primary URL	Active — legacy, higher security

> ***Key implication:*** *Moving to Firebase Hosting as the primary URL removed the infrastructure-level access gate (IAP). The app's login screen is now publicly reachable. Access is still controlled by Firebase Auth + Google OAuth + admin approval, but the security model has shifted from infrastructure blocking to application-layer auth.*

---

1. Authentication

Item	Status	Notes
Google OAuth (Sign in with Google)	✅ Enforced	All users must authenticate via Google before accessing any data
@instrumental.com auto-admin	✅ Implemented	Users with this domain are automatically granted admin role on first sign-in
Non-instrumental users require approval	✅ Implemented	New users land in `pendingUsers` and must be manually approved by an admin
Authorized domains	✅ Configured	Both `deployment-portal-instrumental.web.app` and Cloud Run URL registered in Firebase Auth
OAuth redirect URIs	✅ Configured	`/__/auth/handler` whitelisted in Google Cloud OAuth credentials for Firebase Hosting domain

Risk: Anyone with a Google account can attempt to sign in via the Firebase Hosting URL and will land in the pending queue. An admin must actively approve or reject them. The Cloud Run URL is still protected by IAP and does not expose the login page at all.

---

2. Authorization

Item	Status	Notes
Firebase Database rules	✅ Strong	Server-side rules enforce admin-only writes; deny all by default
`appState/docData`	✅ Admin write only	`role === 'admin'` checked server-side in rules
`appState/projects`	✅ Admin write only	Same as above
`users/` node	✅ Restricted	Only the user themselves or an admin can write; schema validation enforced
`pendingUsers/` node	✅ Admin read only	Only admins can view the pending queue
Client-side role checks	⚠️ Supplementary only	UI hides admin controls for non-admins, but enforcement is at DB rules level
Party-level read isolation	❌ Not enforced at DB level	Any approved user can read all project/party data in Firebase

Risk: There is no per-project or per-party read restriction in the database rules — any approved user can read all project data regardless of their assigned party. This is acceptable if all approved users are trusted, but should be addressed if customers or CMs should not see each other's data.

---

3. Network & Transport Security

Item	Status	Notes
HTTPS on Firebase Hosting	✅ Enforced	Firebase Hosting serves exclusively over HTTPS
HTTPS on Cloud Run	✅ Enforced	Google-managed TLS on the Cloud Run URL
Firebase SDK connections	✅ Secure	All Realtime Database connections use WSS (WebSocket Secure)
IAP on Firebase Hosting	❌ Removed	Firebase Hosting does not support Google Cloud IAP
IAP on Cloud Run	✅ Active	The legacy Cloud Run URL retains full IAP protection
CORS	✅ Not applicable	No custom backend; Firebase SDK handles auth and data directly

Current posture: The primary URL (`web.app`) has no infrastructure-level blocking. The legacy Cloud Run URL retains IAP, which blocks all access before the app loads for users not explicitly granted IAP access.

Recommendation: If a user or stakeholder requires the higher security posture of IAP (e.g. for compliance or enterprise policy), direct them to the Cloud Run URL. Consider mapping a clean custom domain to the

Cloud Run service to make it more accessible while retaining IAP.

---

4. Firebase API Key Exposure

Firebase config (API key, project ID, etc.) is embedded in the client-side JavaScript bundle — this is **expected and normal** for Firebase web apps. The API key is not a secret; it identifies the project. Access is controlled by:

- Firebase Auth (must sign in with Google)
- Firebase Database security rules (enforced server-side)

Action required: Restrict the Firebase API key in Google Cloud Console → APIs & Services → Credentials → API key restrictions → set **HTTP referrers** to:

```
https://deployment-portal-instrumental.web.app/*
https://deployment-portal-901459055521.us-central1.run.app/*
```

This prevents the key from being used from unauthorized domains.

---

5. Input Validation & XSS

Item	Status	Notes
`dangerouslySetInnerHTML`	✅ Not used	No raw HTML injection anywhere in App.jsx
User-supplied text rendered as text nodes	✅ Safe	React escapes all string values by default
Link URLs in milestones	⚠️ Review	Admin-entered URLs are rendered as `<a href>` — no protocol validation in place

Recommendation: Add a guard when saving milestone/document links to ensure the URL starts with `https://` or `http://`, preventing `javascript:` protocol URLs from being stored.

---

6. Data Integrity

Item	Status	Notes
Schema validation on `users/`	✅ Partial	DB rules require `id`, `name`, `email`, `role`, `partyId` fields
Schema validation on `docData/`	❌ None	No server-side schema enforcement on document structure
Audit log	❌ Not present	No record of who changed what data or when

Recommendation: Enable Firebase Realtime Database scheduled exports (Firebase Console → Realtime Database → Backups) to allow point-in-time recovery if data is accidentally corrupted.

---

7. Account & Session Management

Item	Status	Notes
Session persistence	✅ Firebase default	Sessions persist until sign-out or token expiry (~1 hour, auto-refreshed)
Sign out	✅ Available	Sign-out button present in all views
Token revocation on role change	⚠️ Delayed	If a user is removed or demoted, their active session persists until next token refresh

Recommendation: When removing a user's access, also disable their account in Firebase Console → Authentication → Users to immediately invalidate their token.

---

8. Summary of Risks (Priority Order)

Priority	Issue	Recommendation
🔴 High	Login page publicly reachable (no IAP on primary URL)	Use Cloud Run + IAP URL for users requiring infra-level protection; or map a custom domain to Cloud Run
🔴 High	All approved users can read ALL project data	Add per-project or per-party read rules in `database.rules.json`
🟡 Medium	Firebase API key unrestricted	Restrict to both live domains in Google Cloud Console
🟡 Medium	No audit log	Enable Firebase backups; consider Cloud Functions to log writes
🟡 Medium	Token revocation not immediate	Disable accounts in Firebase Auth when removing access
🟢 Low	Milestone/doc link URLs not validated	Add `https://` prefix validation before saving
🟢 Low	Pending user queue not monitored	Set up admin notification when new users request access

---

9. Deployment-Specific Notes

Firebase Hosting URL (primary — lower security tier)

- No IAP. Login screen is publicly reachable by anyone.
- Security relies entirely on Firebase Auth + Google OAuth + admin approval flow.
- Suitable for general use where all stakeholders are known and trusted.

Cloud Run URL (legacy — higher security tier)

- Protected by Google IAP. Unauthorized users see a Google 403 before the app loads.
- Shares the same Firebase database — no data divergence.
- Should be used for users or contexts that require infrastructure-level access control.

- Can be made the primary URL again at any time, or mapped to a custom domain.

---

10. Out of Scope

- Penetration testing / dynamic analysis
- Google Cloud project IAM permissions (who has access to Firebase Console / Cloud Run service)
- Supply chain security (run `npm audit` separately to check dependencies)
- Firebase App Check (not yet implemented — would add an additional layer to prevent unauthorized API usage)